# CISC/CMPE 223 (Software Specifications)

Fall 2025

## Assignment I

**Due Date: Friday, October 10, 2025, 11:59 PM**

Total Marks: **100**

**Instructions:**

1. Theory Answers: Please submit all answers in one document (PDF file format) for the written answers. You can write/type the answers and submit the document. But please make sure that all pages are merged in order/sequentially **(problem** $1$ **to problem** $5$**)** in one PDF file. The name of the pdf file should be **theoryAnswer.pdf**

2. For the programming question, you can choose C or C++ or Java or Python. No other language will be accepted. please make sure that enough comments are given to understand your code and also provide a ReadMe (Markdown or Txt) file explaining how to run your code and how to provide the input for your program.

3. Each programming solution should be in a separate folder. Make the folder name as the question name. For example: program 5 should be in a folder named, $q5$. This folder will contain the read me file related to the question 5 and the code.

4. Finally make the submission folder under your last name and assignment no. For example, if your last name is choudhury, the folder name should be **choudhuryAssignment1**. This folder will have the **theoryAnswer.pdf** and all programming questions folder. Now, compresss this final submission folder, say, **choudhuryAssignment1.zip** and submit it.

## Problem 1

(10 marks)
Draw a **Deterministic Finite Automaton** $M$ that accepts the set of **binary** strings

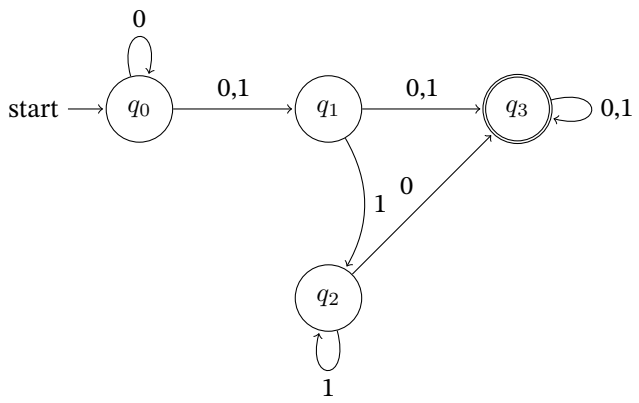
$$S = \{0,\ 11,\ 110,\ 1001,\ 1100,\ 1111,\ 10010,\ 10101,\ 11000,\ 11011,\ 11110, \ldots\},$$

and provide the formal definition $M = \{Q, \sum, \delta, q_0, F\}$ and the transition table.

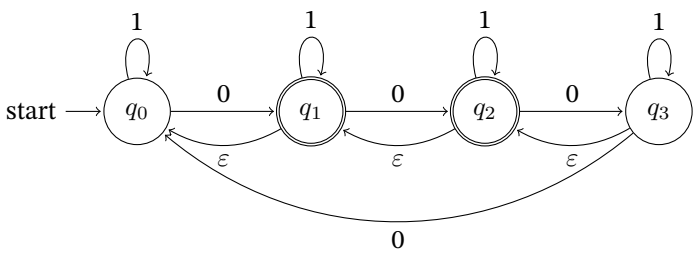
## Problem 2

($15 \times 2 = 30$ marks)
Convert the given **Non-deterministic Finite Automaton** to **Deterministic Finite Automaton** step by step. Draw the state diagram of the **Deterministic Finite Automaton** and write down the formal definition $M = \{Q, \Sigma, \delta, q_0, F\}$ and the transition table.

**(a)**




**(b)**




## Problem 3

($3 + 7 = 10$ marks)
Let $M$ be a (complete) **Deterministic Finite Automaton** with $n$ states. What is the maximum number of distinct languages that can be recognized by modifying only the set of accept states of $M$? Prove your answer.

## Problem 4

$(10 + 10 + 5 = 25$ marks)

*For part (a), the phrase "all strings" refers specifically to strings whose lengths do not exceed the maximum string length given in the table for question (c). Please use this constraint when designing the* ***Deterministic Finite Automaton****.*

Answer the following questions:

(a) Design and draw a **Deterministic Finite Automaton** over the alphabet {a, b} that accepts **all strings** where the absolute difference between the number of a's ($n_a$) and b's ($n_b$) is at most 3, i.e., $|n_a - n_b| \leq 3$.

(b) Write a simulator function that simulates the state transitions of your **Deterministic Finite Automaton** to accept or reject strings.

(c) Complete the table below by running your simulator on the given input strings and determining whether each string is accepted or rejected.

| **Input String** | **Accept/Reject** |
|---|---|
| aaa | |
| abab | |
| aaaabbb | |
| aaaaaaa | |
| bbbbbbb | |
| aaaaaaabbbbbbbb | |
| bbbb | |
| aaaabbbbaaaa | |
| bbbaaaabbbb | |
| aaaaaaaabbbbbbbb | |
| abababab | |
| ababababab | |
| aaabbbbaa | |
| abbbaaaa | |
| bbbaaabbb | |
| (empty string) | |
| a | |
| b | |
| aaabbb | |
| abababababab | |

## Problem 5

(25 marks)
Write a program (in C, C++, Java, or Python) that takes two **Deterministic Finite Automata** $M$ and $M'$ (each in the 5-tuple format) over the same alphabet and searches, by increasing string length, for the **shortest, non-zero length** string $x$ such that exactly one of $M$ or $M'$ accepts $x$.

- Iterate strings in order of non-decreasing length and within each length use the alphabet in sorted order (e.g., for {0,1}: 0,1, then 00,01,10,11, etc.).
- If a disagreement is found, print the string as the answer and stop.
- Stop early and print **Inconclusive** if you exceed a fixed search cap (max_iters or max_length). You do not have to prove equivalence.
- Assume **Deterministic Finite Automata** are complete. If a transition is missing, treat it as an implicit move to a single non-accepting sink state.
- If multiple shortest disagreements of the same length exist, printing any one of them is correct.

**Example input format:**

```
# DFA M: accepts strings ending with '01'
dfa_M = {
    "states": {"m0","m1","m2"},
    "alphabet": {"0","1"},
    "transitions": {
        ("m0","0"): "m1", ("m0","1"): "m0",
        ("m1","0"): "m1", ("m1","1"): "m2",
        ("m2","0"): "m1", ("m2","1"): "m0",
    },
    "start_state": "m0",
    "accept_states": {"m2"},
}

# DFA M': accepts strings ending with '10'
dfa_Mp = {
    "states": {"p0","p1","p2"},
    "alphabet": {"0","1"},
    "transitions": {
        ("p0","0"): "p1", ("p0","1"): "p0",
        ("p1","0"): "p1", ("p1","1"): "p2",
        ("p2","0"): "p1", ("p2","1"): "p0",
    },
    "start_state": "p0",
    "accept_states": {"p1"},  # '10' ends in state p1
}

# Search caps (you may choose either/both)
max_iters = 5000      # stop after this many strings tested
max_length = 20       # or stop at this length
```

**Example Output Format:**

Shortest disagreement: 01